

Writing the Workflow Spec II: Gates, Failure Handling, and Done

FRIDAY AI Education — Episode 033 · 2026-07-19 · Printable Companion

Episode 033 · AI-Native Operator Arc · Lesson 19 of 37

Unit: Workflow Specification, Part 2 of 2

Abstract

The previous paper established the front half of a workflow specification: the objective, the inputs, and the output contract. That half describes what a workflow does when everything goes well. This paper completes the specification by describing everything else — which, in production, is most of what matters. Four elements remain: the **approval boundary** (where the workflow must stop and ask a human), **failure handling** (what happens when a step breaks, and what the workflow is allowed to do about it), **observability** (how you can tell what the workflow did and why, after the fact), and the **definition of done** (the explicit condition under which the workflow is finished, as opposed to merely stopped). Together with Part I, these six elements form a complete, testable specification — the document that turns "an agent that handles X" from a wish into an operable system. We work through all four using three running examples: a Regulatory Radar, a podcast generator, and an email draft queue.

1. Where We Left Off

Part I of this unit gave you the first three sections of a workflow spec:

1. **Objective** — the outcome the workflow exists to produce, stated so a stranger could judge success.
2. **Inputs** — what the workflow consumes, where it comes from, and what happens when it's missing or malformed.
3. **Output contract** — the precise shape, destination, and quality bar of what the workflow produces.

That is the optimistic half of the document. It describes the workflow on its best day. If you stopped there, you would have something that reads like a spec but behaves like a hope: it tells you what the system does when the inputs are clean, the tools respond, and the model behaves. None of those conditions hold reliably, and the earlier lessons on failure modes — hallucinated actions, silent drift, unsafe execution — told you exactly why.

Today's material is the pessimistic half, and it is the half that separates workflows you can actually run from workflows you babysit. The four remaining sections answer four questions:

- **Where does the machine stop and the human decide?** (Approval boundary)
- **When a step fails, what happens next?** (Failure handling)
- **How do you know what actually happened?** (Observability)
- **When is it finished — not stopped, finished?** (Definition of done)

A useful way to hold this: Part I specified the workflow's *behavior*. Part II specifies its *governance*. And governance is not bureaucratic overhead layered on top of the interesting part. In an AI-native company, governance *is* the

interesting part, because the generation is increasingly commoditized and the trust is not.

2. The Approval Boundary

2.1 The core idea

An **approval boundary** is the explicitly drawn line in a workflow beyond which no action executes without human sign-off. Everything on one side of the line, the workflow may do autonomously. Everything on the other side, it may only *propose*.

The critical word is *explicitly*. Every workflow has an approval boundary whether you drew one or not. If you never thought about it, the boundary sits wherever your integration happens to put it — usually at "the agent can do whatever the API key allows," which is not a boundary so much as a blast radius. Drawing the line deliberately is the difference between a governed system and a lucky one.

2.2 What determines where the line goes

The boundary is not a function of how smart the model is. It is a function of two properties of the *action*:

Reversibility. Can the action be undone cheaply? Writing a draft to a folder is reversible — delete the draft. Sending an email is not — you cannot unsend it, and the recipient's impression of you is now updated. Publishing a podcast episode is somewhere in between: technically retractable, reputationally not.

Blast radius. Who and what does the action touch? An action confined to the workflow's own workspace touches nothing. An action that messages a customer, moves money, files a document with a regulator, or publishes under your name touches the world.

The rule that falls out of these two properties is compact:

Irreversible or externally visible actions sit outside the boundary. Reversible, internal actions sit inside it.

Notice what this rule does *not* say. It does not say "risky steps need approval," because "risky" is a feeling and feelings don't compile. It says: classify each action in the workflow by reversibility and blast radius, and the classification tells you which side of the line it lives on. The spec then records that classification, action by action. This is the same discipline the 12-Factor Agents material formalizes as "contact humans with tool calls" — the request for approval is not an exception path bolted on; it is a first-class step the workflow can take, with the same structure and logging as any other step.

2.3 The three examples

Email draft queue. The workflow reads incoming mail, classifies it, and writes reply drafts into a queue. Every generative step — reading, classifying, drafting, even drafting badly — is inside the boundary, because a draft in a queue touches nobody. The single action outside the boundary is *send*. This is why the draft-queue pattern is such a good first automation: the architecture itself is the approval boundary. The human reviewing the queue *is* the gate, and the cost of a bad output is one deleted draft.

Podcast generator. Research, scripting, audio generation, artwork — all internal, all reversible, all inside the boundary. *Publish to the feed* is outside. Note a subtlety: if the generator also emails guests or posts clips to social channels, each of those is a separate externally visible action and each needs its own gate decision. A workflow does not have one approval boundary as a vague perimeter; it has a per-action classification that happens to form a line.

Regulatory Radar. This workflow monitors regulatory publications, filters for relevance, summarizes changes, and flags items that may require a company response. Almost everything it does is inside the boundary — monitoring and summarizing touch nothing. But the moment it drafts a communication to a regulator, or advises that "no action is required" on a filing deadline, you are outside the boundary in an interesting way: here even an *omission* has blast radius. A missed obligation is an irreversible external event that the workflow caused by staying silent. So the spec for Regulatory Radar puts not just "send" outside the boundary, but also "dismiss as irrelevant" for any item above a defined importance threshold. Approval gates can guard inaction, not just action. This is the kind of asymmetry you only catch by writing the spec.

2.4 The boundary is a dial, not a wall

One more property worth writing into the spec: the boundary should be *movable, on evidence*. When the email drafter has produced 500 drafts and you've edited 2% of them, you have data supporting a narrower gate — perhaps auto-send for one low-stakes category of reply. The spec should say what evidence would justify moving the line and who decides to move it. A boundary that can only be loosened by someone quietly commenting out a check is a boundary that will be loosened by exactly that.

3. Failure Handling

3.1 The core idea

Failure handling is the section of the spec that answers, for each step: *what can go wrong here, and what is the workflow permitted to do about it?* The permission framing matters. Failure handling is not primarily about detection — the earlier failure-modes lessons covered how things break. It is about *policy*: the pre-decided response to each class of breakage.

Absent a policy, a failing automated workflow improvises. And an LLM-driven workflow improvises fluently, which is worse than failing loudly. A crashed script produces a stack trace; an improvising agent produces a plausible-looking output built on a step that silently returned garbage. The failure-handling section exists to replace improvisation with decisions you made while calm.

3.2 The four canonical responses

For any failed step there are exactly four dispositions. The spec's job is to assign one to each failure class.

1. **Retry.** Appropriate for transient faults — a timeout, a rate limit, a flaky fetch. The spec must bound it: how many attempts, with what backoff, and what counts as "the same failure" so you don't retry your way into a loop. An unbounded retry is not resilience; it is a slow outage with logging.
2. **Fallback.** Substitute a degraded but acceptable path. The regulatory feed's API is down — fall back to scraping the public page. A fallback must still satisfy the output contract from Part I, possibly at a lower declared quality tier; if it can't, it isn't a fallback, it's a different failure.

3. **Escalate.** Stop and route the problem to a human, with context. Escalation is the failure-handling twin of the approval boundary: gates ask a human before *success* is allowed to touch the world; escalation asks a human after *failure* has been contained. The spec should name where escalations go and what they must contain — a raw stack trace at 3 a.m. is technically an escalation the way a fire alarm with no address is technically a report.
4. **Abort.** Stop cleanly, produce nothing, record why. Aborting is underrated. For many workflows the honest answer to "the input was malformed" is not heroic recovery but a clean refusal that a human will see. The one rule: an abort must be *loud*. A workflow that quietly produces nothing is indistinguishable from a workflow that was never running — this is exactly the silent-drift failure mode from two lessons ago, manufactured on purpose.

The discipline the 12-Factor material adds here is worth naming: when an error occurs mid-workflow, *compact it into context* — capture what failed, in what state, with what inputs — rather than either hiding it or dumping the entire raw wreckage forward. The error becomes structured information the next step (or the escalated human) can actually use.

3.3 The asymmetry that decides the policy

Which of the four responses is right? For each step, ask: *what does a wrong output cost versus a missing output?*

- **Email draft queue:** a missing draft costs a few minutes of human typing; a wrong draft costs almost nothing because the gate catches it. Policy: retry once, then abort loudly. No fallbacks needed — the human is the fallback.
- **Podcast generator:** a missing episode costs a publishing slot; a wrong episode caught at the gate costs regeneration time. Policy: retries and fallbacks are cheap and worthwhile (regenerate the flawed segment, fall back to a shorter format), because everything is recoverable up to the gate.
- **Regulatory Radar:** here the asymmetry inverts. A *missing* output — a monitored source that silently failed for three weeks — is the expensive case, because the workflow's entire value is the claim "we are watching." Policy: aggressive fallbacks for data collection, and escalation (never silent abort) when a source cannot be read. For the Radar, "I could not check source X today" is itself a required output, not an error to swallow.

Same four tools, three different policies, each derived from the cost asymmetry rather than from temperament. That derivation — not the error-handling code — is what the spec records.

4. Observability

4.1 The core idea

Observability is the property that lets you answer, after the fact and from records alone: *what did the workflow do, with what inputs, producing what outputs, and why?* Not "is it up" — that's monitoring, the smoke detector. Observability is the flight recorder: enough recorded state to reconstruct any run without re-running it.

For AI workflows this is not optional polish, for a reason that doesn't apply to ordinary software: the interesting failures are not crashes. A conventional program fails by stopping; the logs of its death are generated for free. An LLM workflow fails by *completing* — confidently, fluently, wrongly. Nothing crashed when the Regulatory Radar misclassified a consultation paper as irrelevant. The only way you will ever diagnose that miss is if the run recorded what the model saw and what it decided. No trace, no diagnosis; no diagnosis, no improvement; no improvement, no earned trust; no earned trust, and the approval boundary can never move. Observability is what makes the whole system *governable over time* rather than merely governed today.

4.2 What the spec must require

For each run, four records:

1. **Trace** — the sequence of steps taken, including retries, fallbacks, and gate decisions. Which path through the workflow was this?
2. **Inputs and intermediate outputs** — what each step consumed and produced. For LLM steps, that means the actual prompt and the actual response, not a paraphrase. When a draft goes wrong, the question is always "what did the model see?" and the trace either answers it or you're re-running the workflow and hoping the failure reproduces.
3. **Decisions and their stated basis** — every point where the workflow chose among alternatives (relevant/irrelevant, escalate/proceed), with the recorded reason. The reason an LLM gives is not a mechanical explanation of its computation — but it is reviewable, and reviewable beats absent.
4. **Human touchpoints** — who approved what, when, and any edits made at the gate. Gate edits are the highest-grade evaluation data you will ever collect: a human expert correcting real outputs on real inputs, for free, as a side effect of governance.

A useful test for whether a workflow is observable enough: *could a competent person who did not build it read one run's records and explain what happened?* If the answer requires the builder's presence, the workflow has memories, not observability.

4.3 Proportionality

Dry truth: nobody reads logs recreationally. Observability has a cost — storage is the trivial part; the real cost is the discipline of structuring every step so its state can be captured. The spec should scale the requirement to the stakes. The email draft queue can log prompt, draft, and gate outcome and call it a day. The Regulatory Radar, whose value proposition is "we can prove we were watching," needs full traces retained for as long as anyone might ask — for that workflow, the audit trail is not overhead on the product; it arguably *is* the product.

5. The Definition of Done

5.1 The core idea

The **definition of done** is the explicit, checkable condition under which a workflow run is complete. Not "the code finished executing" — *done*, in the sense that the objective from Part I has verifiably been met and no further action is owed by anyone.

This sounds too obvious to need a section. It is the most commonly missing part of every informal automation, and its absence produces a specific, corrosive failure: the workflow that *stops* without being *done*. The podcast generator writes an audio file and exits — but the episode was never published, and nobody notices for four days, because the workflow "ran successfully." Stopped and done look identical from the outside unless the spec defines the difference.

5.2 Done is a checklist over the whole spec

A serviceable definition of done has the form: *run X is done when* —

1. the **output contract** is satisfied (the deliverable exists, in the specified shape, at the specified destination),
2. all **approval gates** on the run's path have been resolved — approved, rejected, or expired-and-escalated, but not *pending*,
3. all **failures** encountered have reached a terminal disposition — recovered, or escalated and acknowledged, but not *retrying* and not silently swallowed,
4. the **record** of the run is written.

Read that list again and notice its structure: it has exactly one clause per section of the spec. The definition of done is where the whole document closes over itself — which is why it must come last, and why it could not have been written in Part I.

Two properties are non-negotiable. Done must be **checkable by a machine** ("the summary is good" is not done; "the summary was delivered to the review channel and approved or auto-expired within 24 hours" is). And done must **cover the unhappy paths**: a run that aborted cleanly, loudly, with its record written *is done* — unsuccessfully, but done. The state you are eliminating is not failure; it is *limbo*. An email draft sitting unreviewed in the queue for three weeks is limbo. A Radar item flagged "needs human judgment" that no human ever judged is limbo — and for that workflow, limbo is the worst outcome available, worse than a wrong answer, because it *looks* handled.

This is also why the spec should define done for the *workflow*, not for the software. The email draft queue's software is done when the draft is queued; the workflow is done when the draft is sent, discarded, or expired. The gap between those two definitions is precisely where unowned work accumulates — and a system that accumulates unowned work is not an operating layer, it's a backlog with better marketing.

6. The Complete Specification

Six sections, two halves:

#	Section	Question it answers	Part
1	Objective	What outcome does this exist to produce?	I
2	Inputs	What does it consume, and what if that's missing?	I
3	Output contract	What exactly does it deliver, where, at what bar?	I
4	Approval boundary	Which actions require a human, and why those?	II
5	Failure handling	For each failure class: retry, fallback, escalate, or abort?	II
6	Observability + Done	What is recorded, and when is a run finished?	II

And the three examples, compressed to their Part II essentials:

	Email draft queue	Podcast generator	Regulatory Radar
Gate	Send	Publish (+ any outbound contact)	Send; <i>and</i> dismissal of high-importance items

	Email draft queue	Podcast generator	Regulatory Radar
Failure policy	Retry once, abort loudly; human is the fallback	Retry/fallback freely; all recoverable pre-gate	Escalate on any source failure; silence is the costly error
Observability	Prompt, draft, gate outcome	Trace + artifacts per stage	Full audit trail; the trail is part of the product
Done	Sent, discarded, or expired — never "in queue"	Published and verified live — not "file written"	Every item dispositioned; every source confirmed checked

The pattern worth internalizing from the table: the four elements are the same everywhere, but their *settings* are derived, workflow by workflow, from reversibility, blast radius, and cost asymmetry. The spec is not a template you fill in; it is a derivation you write down.

7. Why This Document Is Worth More Than the Code

A short argument, because it bears on how you spend your time.

The code that implements a workflow will change constantly — models improve, tools get swapped, prompts get tuned. The spec changes slowly, because it encodes decisions about *the business*: what's reversible, what's visible, what a mistake costs, who decides. That stability is what makes the spec the real asset. Three consequences:

It's the unit of delegation. "Build me the Regulatory Radar" is not a delegable instruction; the six-section spec is. This holds whether the builder is a contractor, a teammate, or an agent. An organization that can produce sharp workflow specs can produce automations on demand; one that can't will produce demos.

It's the unit of trust. Investors, customers, and regulators who ask "but what if the AI gets it wrong?" are asking about sections 4 through 6. Answering with a document rather than reassurance is what being taken seriously looks like.

It's the unit of learning. Because the spec makes gates, failures, and done-states explicit and observable, every run generates evidence — approval rates, edit rates, failure frequencies — against a fixed reference. That evidence is what lets you move boundaries, tighten policies, and retire gates *deliberately*. Workflows without specs also generate experience; they just generate it as anecdotes.

Within an operator system like FRIDAY, this is the precise sense in which the orchestrator is "not merely a chatbot": the orchestrator's job is to hold workflows to their specs — enforce the gates, apply the failure policy, write the trace, check for done — while the LLM inside each step remains what it is, a reasoning engine with limits. The spec is the interface between those two facts.

8. The Limitation: Specs Don't Enforce Themselves

The honest failure mode of everything above: a specification is a document, and documents exert no force. Three specific ways this bites.

The spec-implementation gap. Nothing prevents the implementation from drifting away from the spec — a gate quietly bypassed "temporarily," a retry bound raised during an incident and never lowered. The countermeasure is making the spec *executable* wherever possible: the approval gate should be a structural property (drafts physically

land in a queue that has no send capability) rather than a behavioral instruction ("the agent should ask first"). Prefer boundaries the system *cannot* cross to boundaries it is *told* not to cross. Instructed restraint is a prompt; structural restraint is an architecture.

Rubber-stamp decay. An approval gate is only as good as the attention at it. When 98% of items are fine, the human stops reading and starts clicking, and the gate becomes theater — arguably worse than no gate, since it now manufactures false assurance. Countermeasures: keep per-gate volume low, make the reviewable diff small, and treat a sustained high approval rate as evidence for *narrowing the gate's scope* rather than as a reason to keep a bored human in the loop. A gate someone actually attends beats three gates everyone skims.

Enumeration limits. Failure handling covers the failure classes you enumerated. Reality is not obligated to respect your enumeration. The defense is not a longer list — it's a default disposition for *unclassified* failures (escalate, loudly) plus observability good enough that novel failures become named failure classes in the next spec revision. The spec is a living document with a version history, not a stone tablet.

None of these limitations argue against the spec. They argue for the next layer: mechanisms that *enforce* what the spec declares. Which is exactly where this course goes next — tools are how a workflow stops being text and starts being action, and every question in this paper (what may it do, what if the action fails, what gets recorded) becomes concrete at the moment of a tool call. That is **Tool Use I: From Text Generator to Actor**.

Vocabulary

Approval boundary. The explicitly specified line in a workflow separating actions the system may take autonomously from actions it may only propose for human sign-off. Placed by classifying each action's reversibility and blast radius.

Blast radius. The scope of people, systems, and obligations an action affects. Internal-only actions have near-zero blast radius; externally visible ones (send, publish, file) do not.

Reversibility. Whether and how cheaply an action can be undone. With blast radius, one of the two properties that determine an action's side of the approval boundary.

Failure handling. The spec section assigning each anticipated failure class one of four dispositions: retry (bounded), fallback (degraded but contract-satisfying), escalate (route to a human with context), or abort (stop cleanly and loudly).

Escalation. A failure response that halts the affected work and routes the problem, with sufficient context, to a named human channel. The failure-side twin of the approval gate.

Observability. The property that a run's behavior can be reconstructed from its records alone: trace, inputs/outputs per step (actual prompts and responses for LLM steps), decisions with stated reasons, and human touchpoints.

Trace. The recorded sequence of steps a specific run actually took, including retries, fallbacks, and gate outcomes.

Definition of done. The explicit, machine-checkable condition under which a run is complete: output contract satisfied, all gates resolved, all failures terminally dispositioned, record written. Distinguishes *done* from merely *stopped*.

Limbo. The unfinished-but-unowned state a definition of done exists to eliminate: work that has stopped progressing without reaching any terminal disposition.

Rubber-stamping. The decay mode of an approval gate in which the human approves without genuine review, converting a control into theater.

Reading Guide

12-Factor Agents — <https://github.com/humanlayer/12-factor-agents>

A practitioner's catalog of principles for building LLM software reliable enough for production, written from the scars of real deployments. You do not need the whole thing this week. Read with today's four concepts as your lens:

- **Factor 7 — "Contact humans with tool calls"** is the approval boundary made concrete: requesting human input is a first-class, structured step in the workflow, not an exception path. Read this one closely; it is the strongest single anchor for today's material.
- **Factor 8 — "Own your control flow"** explains *where* gates and failure policies live: in orchestration code you control, not inside the model's improvisation. This is the structural-versus-instructed-restraint distinction from Section 8.
- **Factor 9 — "Compact errors into context window"** is failure handling from the model's side: errors as structured, usable information rather than hidden state or raw wreckage.
- **Factor 6 — "Launch/Pause/Resume with simple APIs"** is the machinery that makes approval gates practical: a workflow that cannot pause cannot wait for a human.

Skim the rest for orientation; Factor 10 (small, focused agents) will resonate with why we spec one workflow at a time. Note what the document *is*: engineering folklore hardened into principles, not peer-reviewed research. Its authority comes from matching your own scars — check it against the failure modes from lessons 17 and 18 as you read.

Walking Questions

1. Take a workflow you currently do by hand — any recurring one. Where would you draw its approval boundary, and can you defend the placement of every action using only reversibility and blast radius, without using the word "risky"?
2. For that same workflow: which is more expensive, a wrong output or a missing output? What failure policy does that asymmetry imply — and is it the policy you would have chosen by instinct?
3. The Regulatory Radar gates *dismissal*, not just action. Where else in your operations is inaction an externally consequential event that deserves a gate?
4. What is one bounded workflow where you could test a full six-section spec safely this month — small enough that limbo, if it occurs, costs you nothing but the lesson?
5. Honest check: which of the four elements — gates, failure handling, observability, done — could you now explain to a skeptical engineer without notes? Whichever one you'd hesitate on is the paragraph to reread.

One-Sentence Takeaway

A workflow specification is complete only when it says where the machine must stop and ask, what it does when steps fail, how you can reconstruct what it did, and exactly what *finished* means — because everything before that describes a workflow's best day, and systems earn trust on their worst.